

Quaint – interaktywny symulator dynamiki kwantowej w dowolnym skończonym potencjale 2D z interfejsem graficznym

Szymon Borowski¹, Franciszek Hansdorfer¹, Bartosz Łakomy¹,
Adam Podkowiński¹ oraz Jacek Winiarczyk^{†1}

Opiekun projektu: mgr Bartosz Kasza¹

¹Wydział Fizyki, Uniwersytet Warszawski

Czerwiec 2026

Streszczenie

W pracy przedstawiono *Quaint* – symulator dynamiki kwantowej w dowolnym skończonym potencjale 2D. Aplikacja numerycznie rozwiązuje zależne od czasu równanie Schrödingera, wykorzystując niejawną metodę Cranka-Nicolson (z jednostkową faktoryzacją LU operatora Cayleya) oraz jawną metodę Split-Step Fourier opartą na dyskretnej transformacji sinusowej (DST-I), co wymusza spełnienie jednorodnych warunków brzegowych Dirichleta. Silnik oparty na OpenGL renderuje funkcję falową w 3D, gdzie wysokość powierzchni odpowiada gęstości prawdopodobieństwa, a kolor reprezentuje fazę. Program zintegrowano z interaktywnym edytorem potencjałów, mechanizmem kontroli stabilności numerycznej wykorzystującym warunki Nyquista i Couranta oraz modułem automatycznych testów z rozwiązaniami analitycznymi.

[†]e-mail: jk.winiarczy@student.uw.edu.pl

Spis treści

1	Wstęp	3
1.1	Wstęp teoretyczny	3
1.2	Propagator numeryczny	3
2	Metody numeryczne wyznaczania ewolucji	3
2.1	Metoda Cranka-Nicolson	4
2.1.1	Splaszczanie funkcji falowej	4
2.1.2	Konstrukcja operatora Hamiltona przez iloczyn Kroneckera	4
2.1.3	Metoda Cayleya	5
2.2	Metoda Split-Step Fouriera (SSFM)	5
2.2.1	Warunki brzegowe i Dyskretna Transformacja Sinusowa (DST)	5
2.2.2	Reprezentacja operatora energii kinetycznej	6
2.2.3	Symetryczna metoda SSFM	6
2.3	Porównanie CN i SSFM	6
3	Parametry symulacji i konfiguracja	7
4	Reprezentacja stanu początkowego	7
5	Przestrzeń potencjału	8
6	Wczytywanie i ustawianie symulacji	8
6.1	Ustawienia symulacji i rysowanie potencjału	8
6.2	Integracja z backendem	9
7	Rendering 3D symulacji	9
7.1	Interpolacja i optymalizacje renderingu	10
7.2	Sterowanie symulacją	10
8	Porównanie symulacji z rozwiązaniem analitycznym	11
8.1	Rozwiązania analityczne	11
8.2	Metodologia walidacji i metryki błędu	11
8.3	Analiza zbieżności schematów numerycznych	12
8.4	Automatyzacja procesów testowych	12
9	Repozytorium kodu oraz dalszy rozwój	13
	Bibliografia	13
	Dodatek	13
A	Podręcznik użytkownika w języku angielskim	13
B	Podział prac w zespole	26

1 Wstęp

W dydaktyce fizyki bardzo przydatnym narzędziem są wizualizacje. Niestety w wielu podręcznikach i innych pomocach naukowych traktujących o mechanice kwantowej wizualizacji jest bardzo mało, a jeżeli już są, to sprowadzają się do rozwiązywania prostych, wysoce symetrycznych przypadków. W celu rozwiązania tego problemu zdecydowano się na stworzenie symulatora ewolucji funkcji falowej dla dowolnego, zdefiniowanego przez użytkownika, potencjału. Taki interaktywny symulator byłby wygodnym narzędziem dydaktycznym pozwalającym na lepsze zrozumienie równania Schrödingera.

1.1 Wstęp teoretyczny

Równanie Schrödingera dla potencjału niezależnego od czasu jest postaci:

$$\hat{H}\psi(\vec{r}, t) = (\hat{T} + \hat{V})\psi(\vec{r}, t) = \left(-\frac{\hbar^2}{2m}\nabla^2 + V(\vec{r})\right)\psi(\vec{r}, t) = i\hbar\frac{\partial}{\partial t}\psi(\vec{r}, t) \quad (1)$$

Rozwiązanie powyższego równania można zapisać jako:

$$\psi(\vec{r}, t) = e^{-i\hat{H}t/\hbar}\psi(\vec{r}, 0) = e^{-i(\hat{T}+\hat{V})t/\hbar}\psi(\vec{r}, 0) \quad (2)$$

gdzie $\exp(-i\hat{H}t/\hbar)$ oznaczamy jako $\hat{U}(t)$, jest ono propagatorem ewolucji. Operatory $\hat{T}$, $\hat{V}$ są kolejno operatorami energii kinetycznej i potencjalnej. Ułatwieniem obliczeń jest fakt, że operator $V(\vec{r})$ nie zależy od czasu.

W przypadku operatorów niekomutujących (inaczej nieprzemiennych) tożsamość $\exp(\hat{X} + \hat{Y}) = \exp(\hat{X})\exp(\hat{Y})$ nie zachodzi. Wyrażenie to można przybliżyć korzystając z twierdzenia Bakera–Campbella–Hausdorffa: $\exp(\hat{X})\exp(\hat{Y}) = \exp(\hat{Z})$, gdzie $Z = X + Y + \frac{1}{2}[X, Y] + \frac{1}{12}[X, [X, Y]] + \frac{1}{12}[Y, [Y, X]] + \dots$, przy czym $[X, Y]$ oznacza komutator.

1.2 Propagator numeryczny

Dla małego kroku czasowego Δt funkcja falowa jest dana wzorem:

$$\psi(\vec{r}, t + \Delta t) = \hat{U}(\Delta t)\psi(\vec{r}, t) \quad (3)$$

Rozwinięcie propagatora w szereg Taylora nie jest optymalnym podejściem, ponieważ nie zachowuje ono unitarności ($\int_{\Omega} |\psi|^2 d^2r = 1$). Z tego względu konieczne jest zastosowanie bardziej zaawansowanych metod aproksymacji.

2 Metody numeryczne wyznaczania ewolucji

Centralnym elementem symulatora jest moduł `Solver.py`, dedykowany rozwiązywaniu zależnego od czasu równania Schrödingera. Implementuje on mechanizm automatycznej kontroli stabilności numerycznej w oparciu o twierdzenie Nyquista-Shannona (ograniczające maksymalny dopuszczalny pęd cząstki, gdzie k jest liczbą falową)

$$|\vec{k}| \ll k_{max} = \frac{2\pi}{2\Delta x} \quad (4)$$

oraz warunek Couranta–Friedrichsa–Lewego, wyznaczający bezpieczną długość kroku czasowego.

$$C = \frac{|\vec{k}|}{m} \frac{\Delta t}{\Delta x} \ll 1 \quad (5)$$

W oprogramowaniu wykorzystano dwa główne podejścia numeryczne do propagacji: metodę Cranka-Nicolson oraz warianty metody podziału operatora ewolucji (ang. *Split-Step Fourier Method*). Do obliczeń wykorzystano biblioteki NumPy [1] oraz SciPy [2] do tworzenia wykresów użyto Matplotlib [3].

2.1 Metoda Cranka-Nicolson

Metoda ta jest algorytmem niejawnym (szukana wartość rozwiązania w nowym kroku jest uwikłana po obu stronach równania), wynikającym z przybliżenia unitarnego operatora ewolucji transformacją Cayleya. Wyznaczenie funkcji falowej w kolejnym kroku czasowym wymaga rozwiązania układu równań liniowych [4]:

$$\left(I + \frac{i\Delta t}{2\hbar}\hat{H}\right)\psi(t + \Delta t) = \left(I - \frac{i\Delta t}{2\hbar}\hat{H}\right)\psi(t) \quad (6)$$

gdzie $\hat{H}$ to dyskretny operator Hamiltona, składający się z członu kinetycznego (opartego na dyskretnym operatorze Laplace'a z krokami $\Delta x, \Delta y$) oraz diagonalnego członu potencjalnego. Dwuwymiarowy operator Laplace'a L_{2D} budowany jest przy użyciu iloczynu Kroneckera jednowymiarowych, trójpunktowych operatorów różnicowych.

2.1.1 Spłaszczanie funkcji falowej

W celu zastosowania narzędzi algebry liniowej macierz $\psi_{i,j}$ należy zamienić na wektor Ψ_m . Przestrzeń ciągła jest odwzorowywana na dyskretną siatkę węzłów o wymiarach $N_x \times N_y$ z krokami Δx i Δy . Wartość pola w punkcie $(x_i, y_j) = (i\Delta x, j\Delta y)$ w chwili $t_n = n\Delta t$ reprezentuje element $\psi_{i,j}^n$, gdzie $i \in \{0, \dots, N_x - 1\}$ oraz $j \in \{0, \dots, N_y - 1\}$.

Transformacja macierzy $\psi \in \mathbb{C}^{N_x \times N_y}$ w wektor kolumnowy $\Psi \in \mathbb{C}^{N_x N_y}$ przebiega wierszowo:

$$\Psi_m^n = \psi_{i,j}^n, \quad m = i \cdot N_y + j \quad (7)$$

2.1.2 Konstrukcja operatora Hamiltona przez iloczyn Kroneckera

Jednowymiarowe operatory różnicowe drugiego rzędu $D_{xx} \in \mathbb{R}^{N_x \times N_x}$ oraz $D_{yy} \in \mathbb{R}^{N_y \times N_y}$, przy założeniu jednorodnych warunków brzegowych Dirichleta ($\psi|_{\partial\Omega} = 0$) przyjmują postać macierzy trójdzielnych [5]:

$$D_{xx} = \frac{1}{\Delta x^2} \begin{pmatrix} -2 & 1 & 0 & \dots & 0 \\ 1 & -2 & 1 & \dots & 0 \\ 0 & 1 & -2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & -2 \end{pmatrix}, \quad D_{yy} = \frac{1}{\Delta y^2} \begin{pmatrix} -2 & 1 & 0 & \dots & 0 \\ 1 & -2 & 1 & \dots & 0 \\ 0 & 1 & -2 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & -2 \end{pmatrix} \quad (8)$$

Dwuwymiarowy dyskretny operator Laplace'a $L_{2D} \in \mathbb{R}^{N_x N_y \times N_x N_y}$ określa się jako sumę Kroneckera:

$$L_{2D} = D_{xx} \oplus D_{yy} = D_{xx} \otimes I_y + I_x \otimes D_{yy} \quad (9)$$

gdzie $\otimes$ to iloczyn Kroneckera, a I_x, I_y to macierze jednostkowe. Działanie jego składowych opisują relacje:

$$[(D_{xx} \otimes I_y)\Psi]_{iN_y+j} = \sum_{k=0}^{N_x-1} (D_{xx})_{i,k} \psi_{k,j} = \frac{\psi_{i+1,j} - 2\psi_{i,j} + \psi_{i-1,j}}{\Delta x^2} \quad (10)$$

$$[(I_x \otimes D_{yy})\Psi]_{iN_y+j} = \sum_{l=0}^{N_y-1} (D_{yy})_{j,l} \psi_{i,l} = \frac{\psi_{i,j+1} - 2\psi_{i,j} + \psi_{i,j-1}}{\Delta y^2} \quad (11)$$

Ostatecznie, dyskretny hamiltonian $H \in \mathbb{C}^{N_x N_y \times N_x N_y}$ wyraża się sumą:

$$H = -\frac{\hbar^2}{2m} L_{2D} + \text{diag}(\mathbf{V}) \quad (12)$$

gdzie $\mathbf{V}$ to spłaszczony wektor potencjału $\mathbf{V}_{iN_y+j} = V(x_i, y_j)$.

2.1.3 Metoda Cayleya

Przybliżenie pochodnej czasowej przy pomocy symetrycznego schematu różnicowego (metoda trapezów) w punkcie $t_{n+1/2}$ prowadzi do schematu Cranka-Nicolson [6]:

$$i\hbar \frac{\Psi^{n+1} - \Psi^n}{\Delta t} = \hat{H} \left(\frac{\Psi^{n+1} + \Psi^n}{2} \right) \quad (13)$$

Co po przekształceniu daje równanie 6:

$$\left(I + \frac{i\Delta t}{2\hbar} \hat{H} \right) \Psi^{n+1} = \left(I - \frac{i\Delta t}{2\hbar} \hat{H} \right) \Psi^n \quad (14)$$

Powyższe równanie można zapisać w postaci równania macierzowego $A\Psi^{n+1} = B\Psi^n$. Sprowadza to cały problem do znalezienia Ψ^{n+1} w równaniu $A\Psi^{n+1} = b$. Powstały rzadki układ równań rozwiązywany jest za pomocą rozkładu LU (substytucji wstecznej), wykonywanej jednorazowo dla danego potencjału z wykorzystaniem biblioteki `scipy.sparse.linalg.factorized`, co optymalizuje obliczenia dla kolejnych iteracji. Sprowadza to każdy krok czasowy do substytucji wstecznej, która ma złożoność $\mathcal{O}(N_x N_y)$. Z wykorzystaniem przybliżenia Padé otrzymujemy propagator postaci:

$$e^{-i\hat{H}\Delta t/\hbar} = \left(1 + i\hat{H}\Delta t/2\hbar \right)^{-1} \left(1 - i\hat{H}\Delta t/2\hbar \right) + \mathcal{O}(\Delta t^3) \quad (15)$$

2.2 Metoda Split-Step Fouriera (SSFM)

Podczas gdy schemat Cranka-Nicolson operuje na macierzach rzadkich i wymaga rozwiązywania układów równań liniowych, alternatywne podejście zrealizowane w pozostałych modułach opiera się na przybliżonym podziale operatora ewolucji (ang. operator splitting).

Metoda Split-Step Fourier [7] omija problem niekomutujących operatorów $\hat{T}$, $\hat{V}$ poprzez faktoryzację operatora ewolucji na sekwencję operatorów działających niezależnie w przestrzeniach, w których przyjmują one formę diagonalną: operator $\hat{V}$ w przestrzeni położeń, a operator $\hat{T}$ w przestrzeni pędów. Stosując rozkład Suzukiego-Trottera pierwszego rzędu [8] otrzymujemy:

$$\psi(t + \Delta t) = \exp(-i\hat{V}\Delta t) \exp(-i\hat{T}\Delta t) + \mathcal{O}(\Delta t^2) \quad (16)$$

2.2.1 Warunki brzegowe i Dyskretna Transformacja Sinusowa (DST)

Standardowe implementacje metody SSFM często wykorzystują Szybką Transformację Fouriera (FFT), co domyślnie wymusza okresowe warunki brzegowe. Aby zachować spójność z jednorodnymi warunkami Dirichleta $\psi|_{\partial\Omega} = 0$, które wprowadza skończoność potencjału, w omawianej architekturze zastosowano Dyskretną Transformację Sinusową pierwszego rodzaju (DST-I) [9]. Baza w funkcjach sinus naturalnie spełnia założenie o znikaniu funkcji falowej na brzegach dziedziny. Istotnie dla odwrotnej transformacji sinusowej pierwszego rodzaju, funkcja bazowa dla dowolnego wymiaru d jest iloczynem tensorowym funkcji bazowych postaci:

$$\prod_d \sin \left(\frac{\pi k_d n_d}{N_d + 1} \right),$$

więc na brzegach (czyli dla $n_d = 0$, lub $n_d = N_d + 1$) mamy $\sin(0) = 0$ oraz $\sin(\pi k_d) = 0$.

DST-I mapuje dyskretną siatkę przestrzenną (o wymiarach $N_x \times N_y$) na przestrzeń wektorów falowych, redukując błędy numeryczne względem FFT wynikające z niezerowości funkcji falowej na brzegu dziedziny.

2.2.2 Reprezentacja operatora energii kinetycznej

Niech N_x, N_y oznaczają liczbę punktów siatki we współrzędnych odpowiednio x i y . Jak zostało pokazane w 2.1.2 pełny, dwuwymiarowy dyskretny operator Laplace'a L_{2D} konstruuje się jako sumę Kroneckera:

$$L_{2D} = D_{xx} \otimes I_y + I_x \otimes D_{yy}, \quad (17)$$

gdzie I_x oraz I_y to macierze jednostkowe odpowiednich wymiarów. Wartości własne ujemnego operatora Laplace'a ($-L_{2D}$) wynoszą [10]:

$$\lambda^{a,b} = \frac{4}{\Delta x^2} \sin^2\left(\frac{\pi a}{2L}\right) + \frac{4}{\Delta y^2} \sin^2\left(\frac{\pi b}{2M}\right), \quad (18)$$

przy czym $0 < a < L$, $0 < b < M$, gdzie $L = N_x + 1$, $M = N_y + 1$.

Zakładając jednostki $\hbar = 1$, w bazie wektorów własnych operator T jest postaci:

$$\hat{T} = \frac{\lambda^{a,b}}{2m}, \quad (19)$$

Propagator kinetyczny dla kroku czasowego Δt , daje się w rezultacie zapisać jako:

$$\hat{U}_T = \exp(-iT\Delta t) \quad (20)$$

2.2.3 Symetryczna metoda SSFM

Aby zredukować błąd dyskretyzacji wynikający z tego, że operatory $\hat{T}$ i $\hat{V}$ nie komutują zaimplementowano również „symetryczny” wariant metody SSFM. Polega on na rozdzieleniu propagacji na trzy etapy:

1. Pół-krok w przestrzeni położeń: $\psi \leftarrow \psi \cdot U_{V/2}$.
2. Pełen krok w przestrzeni pędów po transformacji DST: $\psi_k \leftarrow \psi_k \cdot U_T$ i odwrotna DST.
3. Kolejny pół-krok w przestrzeni położeń: $\psi \leftarrow \psi \cdot U_{V/2}$.

Wówczas stosując rozkład Suzukiego-Trottera drugiego rzędu [8] dostajemy:

$$\exp(-i(T + V)\Delta t) = \exp\left(-iV\frac{\Delta t}{2}\right) \exp(-iT\Delta t) \exp\left(-iV\frac{\Delta t}{2}\right) + O(\Delta t^3). \quad (21)$$

2.3 Porównanie CN i SSFM

Tabela 1: Zestawienie porównawcze właściwości numerycznych metody Cayleya (schematu Cranka-Nicolson) oraz rozwinięcia Suzukiego-Trottera (SSFM).

Cecha	Cranka-Nicolson (CN)	Split-Step Fourier (SSFM)
Typ metody	Niejawna	Jawna

Podstawa matematyczna	Transformacja Cayleya (przybliżenie wymierne Padé)	Rozkład Suzukiego-Trottera (podział operatora)
Propagator	$(I + \frac{i\Delta t}{2}\hat{H})^{-1}(I - \frac{i\Delta t}{2}\hat{H})$	Standardowy: $\exp(-i\hat{V}\Delta t)\exp(-i\hat{T}\Delta t)$ Symetryczny: $\exp(-i\hat{V}\frac{\Delta t}{2})\exp(-i\hat{T}\Delta t)\exp(-i\hat{V}\frac{\Delta t}{2})$
Złożoność kroku	Wysoka (faktoryzacja LU, układy równań)	Niska (mnożenie macierzy, transformacja DST/FFT)
Rząd błędu lokalnego	$\mathcal{O}(\Delta t^3)$	Standardowy: $\mathcal{O}(\Delta t^2)$ Symetryczny: $\mathcal{O}(\Delta t^3)$
Główne źródło błędu	Przybliżenie Padé	Brak komutacji operatorów energii
Unitarność	Zachowana	Zachowana

3 Parametry symulacji i konfiguracja

Moduł `Params.py` odpowiada za przechowywanie parametrów fizycznych oraz parametrów numerycznych symulacji. W tym modelu definiujemy strukturę danych przechowującą m.in. wymiary siatki przestrzennej (L_x, L_y), krok siatki ($\Delta x, \Delta y$), dyskretyzację czasową (Δt), całkowity czas symulacji (T_{tot}) oraz masę cząstki (m) i jej parametry początkowe. Klasa `Params` implementuje również mechanizmy eksportu i importu ustawień do formatu JSON, co umożliwia wygodne zapisywanie oraz wczytywanie konfiguracji z plików. Symulacje liczone w jednostkach atomowych (a.u.), gdzie ($\hbar = e^- = m_e = 4\pi\epsilon_0 = 1$).

4 Reprezentacja stanu początkowego

Za reprezentację stanu kwantowego układu odpowiada moduł `StationaryWaveFunc.py`, operujący na zespolonej macierzy funkcji falowej określonej na dwuwymiarowej siatce numerycznej. Moduł oblicza również całkowite prawdopodobieństwo ($P = \int_{\Omega} |\psi|^2 d^2r$), co umożliwia jej poprawną normalizację.

Podstawowym modelem generowanego stanu początkowego jest dwuwymiarowy pakiet falowy, zaimplementowany w klasie `GaussianPacket`. Jego przestrzenny rozkład opisany jest wzorem:

$$\psi(\mathbf{r}) = C \exp\left(i\mathbf{k}_0 \cdot \Delta\mathbf{r} - \frac{1}{2}\Delta\mathbf{r}^T \Sigma_0^{-1} \Delta\mathbf{r}\right) \quad (22)$$

gdzie:

- $\Delta\mathbf{r} = \mathbf{r} - \mathbf{r}_0$ to wektor przesunięcia względem początkowego położenia cząstki $\mathbf{r}_0$,
- $\mathbf{k}_0$ to wektor falowy determinujący początkowy pęd pakietu,
- Σ_0 to macierz kowariancji, definiująca szerokość oraz korelacje przestrzennego rozmycia w różnych kierunkach (przed generacją weryfikowana jest jej dodatnia określoność),
- C jest stałą normalizacyjną, gwarantującą, że całkowite prawdopodobieństwo wynosi 1.

Początkowe parametry pakietu falowego $\vec{k}_0$, Σ_0 , $\vec{r}_0$ są przechowywane w klasie `Params`.

5 Przestrzeń potencjału

Moduł `Potential.py` odpowiada za definiowanie przestrzennego rozkładu potencjału $V(x, y)$. Bazowa klasa `Potential` obsługuje algebrę potencjałów (m.in. dodawanie i odejmowanie), co umożliwia tworzenie złożonych form. Oprogramowanie udostępnia następujące wbudowane profile:

- **Nieskończona studnia potencjału (`InfiniteWellPotential`):** Implementuje obszar o zerowym potencjale, w którym cząstka porusza się swobodnie. Uwięzienie realizowane jest za pomocą warunków brzegowych Dirichleta (funkcja falowa znika na brzegach domeny).
- **Gaussowska bariera potencjału (`GaussianBumpPotential`):** Implementuje zlokalizowaną barierę o profilu rozkładu normalnego, daną równaniem:

$$V(\mathbf{r}) = V_0 \exp\left(-\frac{1}{2}\Delta\mathbf{r}^T \Sigma_0^{-1} \Delta\mathbf{r}\right) \quad (23)$$

- **Oscylator harmoniczny (`HarmonicPotential`):** Modeluje dwuwymiarowy, paraboliczny profil potencjału w postaci $V(x, y) = \frac{1}{2}k(\Delta x^2 + \Delta y^2)$ z minimum w punkcie $\mathbf{r}_0$.
- Ponadto zaimplementowano profile takie jak: bariera w kształcie litery „W” (`WShaped`), studnia w studni „Matryoshka” oraz prostokątnej bariery (`Slab`).

6 Wczytywanie i ustawianie symulacji

6.1 Ustawienia symulacji i rysowanie potencjału

Graficzny interfejs użytkownika został oparty na bibliotece PyQt6. Konfiguracja symulacji została zrealizowana jako osobne okno `SetupDrawer`, które łączy ustawienia numeryczne z interaktywnym edytorem potencjału. Użytkownik może wybrać metodę rozwiązywania równania Schrödingera: Crank–Nicolson, SSFM lub symetryczne SSFM. Oprócz tego ustawiane są: całkowity czas symulacji T_{tot} , krok czasowy Δt , liczba kroków obliczeniowych przypadających na jedną klatkę animacji, rozmiar obszaru symulacji, krok siatki, wysokość ścian potencjału, masa cząstki oraz macierz kowariancji początkowego pakietu falowego.

Potencjał może być tworzony ręcznie lub ładowany z przygotowanych wcześniej ustawień. W trybie ręcznym użytkownik korzysta z pędzla i gumki, a sama kanwa jest przechowywana w rozdzielczości odpowiadającej aktualnej siatce obliczeniowej. Jasność narysowanego obrazu jest później przeliczana na wartość potencjału:

$$V(x, y) = \frac{255 - I(x, y)}{255} V_0, \quad (24)$$

gdzie $I(x, y)$ oznacza jasność piksela, a V_0 jest ustawioną wysokością ścian. Dzięki temu biały kolor odpowiada zerowemu potencjałowi, a ciemniejsze fragmenty tworzą bariery. Dostępne są również predefiniowane profile potencjału opisane w rozdziale dotyczącym przestrzeni potencjału, co pozwala szybko rozpocząć symulację bez ręcznego rysowania całego układu.

W tym samym oknie ustawia się początkowy pakiet falowy. Punkt kliknięcia oznacza położenie początkowe r_0 , a przeciągnięcie myszy wyznacza wektor pędu k_0 . Ponieważ współrzędne interfejsu Qt mają oś Y skierowaną w dół, a model fizyczny używa standardowej orientacji matematycznej, przy zapisie wykonywana jest konwersja współrzędnych. Analogicznie przekalowywany jest wektor pędu, tak aby długość strzałki na kanwie odpowiadała wartości k_0 używanej przez backend.

6.2 Integracja z backendem

Najważniejszym punktem integracji jest moduł `simulation_builders.py`. Odpowiada on za zamianę danych pochodzących z frontendu na obiekty backendu: `Params`, `Potential` oraz odpowiedni solver. Frontend pracuje na danych wygodnych dla interfejsu graficznego, natomiast backend oczekuje macierzy NumPy i parametrów fizycznych w jednostkach siatki lub w jednostkach fizycznych. Dlatego przy przekazywaniu danych wykonywane są m.in.:

- odwrócenie osi potencjału między widokiem użytkownika a macierzą backendu,
- przeskalowanie r_0 przez krok siatki,
- przeskalowanie macierzy σ_0 przez Δx^2 ,
- konwersja typów NumPy do postaci wymaganej przez solver.

Backend przechowuje funkcję falową jako macierz zespoloną typu `complex128`, co zapewnia dokładność obliczeń numerycznych. Po wygenerowaniu klatek animacji dane są jednak konwertowane do `complex64`. Jest to świadoma optymalizacja po stronie frontendu: do renderowania nie jest potrzebna pełna precyzja `complex128`, a zmniejszenie typu danych ogranicza zużycie pamięci i przyspiesza przesyłanie oraz buforowanie klatek.

Parametry symulacji mogą być zapisywane do pliku JSON i odczytywane z pliku. Klasa `Params` przy zapisie zamienia typy `Enum` na napisy, a tablice NumPy na listy, ponieważ JSON nie obsługuje bezpośrednio obiektów NumPy. Przy odczycie wykonywana jest operacja odwrotna: listy są zamieniane z powrotem na tablice `float64`, a napisy określające solver i typ potencjału na odpowiednie wartości enum. Dzięki temu plik zawiera zarówno ustawienia fizyczne, jak i pełną macierz potencjału, jeśli użytkownik narysował własny układ.

7 Rendering 3D symulacji

Rendering został wykonany z użyciem OpenGL poprzez biblioteki `pyqtgraph.opengl` oraz `PyOpenGL`. Scena składa się z dwóch siatek trójkątów: jednej dla funkcji falowej i jednej dla potencjału. Dla każdej siatki przygotowywane są wierzchołki, kolory oraz indeksy trójkątów. Współrzędne x i y opisują położenie na płaszczyźnie symulacji, natomiast współrzędna z jest wyliczana osobno dla fali i potencjału.

Dla funkcji falowej wysokość powierzchni jest proporcjonalna do gęstości prawdopodobieństwa:

$$z_\psi = |\psi(x, y)|^2 \cdot s_\psi, \quad (25)$$

gdzie z_ψ oznacza wysokość siatki funkcji falowej w renderingu, a s_ψ jest ustawianym przez użytkownika współczynnikiem skali pionowej. Kolor koduje fazę funkcji falowej: faza jest zamieniana na odcień w przestrzeni HSV, a jasność zależy od amplitudy. Dzięki temu użytkownik widzi jednocześnie rozkład prawdopodobieństwa oraz informację o fazie funkcji falowej.

Potencjał jest renderowany jako osobna, półprzezroczysta szara powierzchnia:

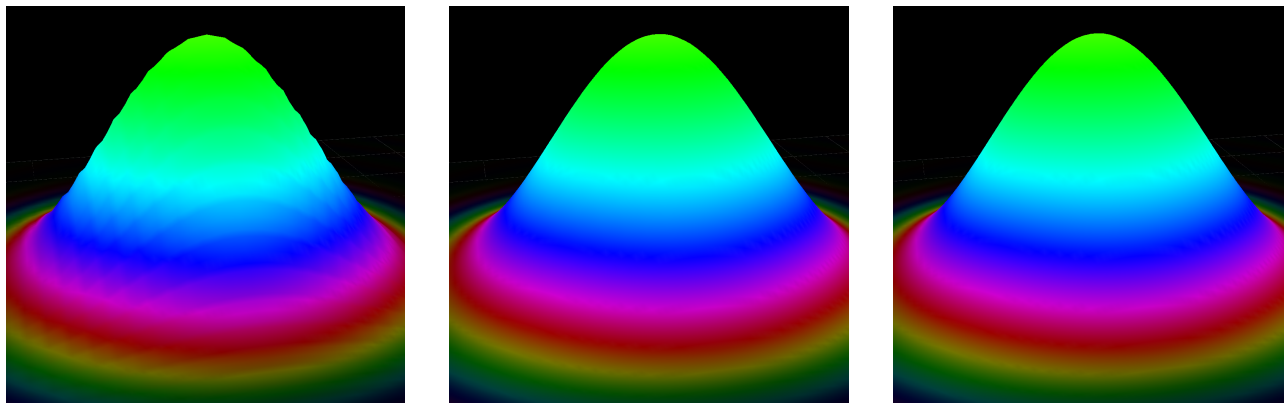
$$z_V = V(x, y) \cdot s_V - z_{\text{offset}}, \quad (26)$$

gdzie z_V oznacza wysokość powierzchni potencjału, $V(x, y)$ jest wartością potencjału w danym punkcie, s_V jest współczynnikiem skali pionowej potencjału, a z_{offset} pozwala przesunąć powierzchnię potencjału w dół lub w górę względem funkcji falowej. Użytkownik może zmieniać skalę wysokości fali, skalę potencjału, przesunięcie potencjału, jasność fali oraz przezroczystość potencjału bez ponownego liczenia symulacji.

7.1 Interpolacja i optymalizacje renderingu

Siatka obliczeniowa backendu jest stosunkowo rzadka, ponieważ jej zagęszczenie bezpośrednio wpływa na koszt symulacji. Do renderingu używana jest więc gęstsza siatka wizualna. Dla funkcji falowej interpolowane są osobno część rzeczywista i urojona, a następnie na tej podstawie wyznaczone są faza, amplituda i gęstość prawdopodobieństwa. Zaimplementowano interpolację B-spline z konfigurowalnym rzędem wielomianu.

Porównano trzy najważniejsze warianty interpolacji funkcji falowej: liniową, kwadratową i sześcienną. Domyślnie wybrano interpolację kwadratową, czyli rząd 2. Daje ona dobry kompromis między jakością obrazu i czasem renderowania: jest znacznie gładzsza od interpolacji liniowej, a jednocześnie szybsza i mniej podatna na przeregulowania niż interpolacja sześcienna.



a) rząd 1

b) rząd 2

c) rząd 3

Rysunek 1: Porównanie interpolacji funkcji falowej dla wielomianów różnych rzędów. Jako domyślną metodę wybrano interpolację kwadratową, czyli rząd 2.

Potencjał interpolowany jest inaczej niż funkcja falowa. Zastosowano interpolację dwuliniową, realizowaną przez `RectBivariateSpline` przy użyciu wielomianów pierwszego stopnia w obu kierunkach przestrzennych. Wybrano ją dlatego, że potencjał często zawiera ostre bariery i krawędzie. Interpolacja wyższego rzędu mogłaby tworzyć nienaturalne oscylacje przy ścianach potencjału, dlatego po interpolacji wartości są dodatkowo ograniczane do zakresu wartości z oryginalnej siatki.

W rendererze zastosowano kilka optymalizacji. Geometria siatki, indeksy trójkątów i bufor kolorów są przygotowywane wcześniej, a nie tworzone od zera przy każdej klatce. Klatki funkcji falowej są cache'owane po pierwszym przeliczeniu na dane renderera, co przy przewijaniu animacji pozwala uniknąć ponownej interpolacji. Obliczenia koloru HSV do RGB są wektoryzowane w NumPy i korzystają z wcześniej zaalokowanych buforów pomocniczych. Dodatkowo sama symulacja liczona jest w osobnym wątku, więc interfejs pozostaje responsywny.

7.2 Sterowanie symulacją

Użytkownik może kontrolować zarówno przebieg animacji, jak i sposób jej wyświetlania. Przycisk `Play/Pause` uruchamia lub zatrzymuje odtwarzanie, suwak czasu pozwala przejść do dowolnej klatki, a etykieta czasu pokazuje aktualny czas fizyczny w jednostkach atomowych. Dostępne są też skróty klawiaturowe: spacja przełącza odtwarzanie, a strzałki w lewo i w prawo przesuwać animację o większą liczbę klatek.

Przycisk `Reset Camera` przywraca domyślne ustawienie kamery 3D. Przycisk `Hide/Show Potential` ukrywa lub pokazuje powierzchnię potencjału, co ułatwia analizę samej funkcji falowej. `Simulation Setup` otwiera okno konfiguracji fizycznej i powoduje ponowne przeliczenie

symulacji po zatwierdzeniu zmian. **Settings** zmienia wyłącznie parametry wizualne, takie jak FPS, skala wysokości, jasność, przezroczystość potencjału oraz rząd interpolacji. Podczas długich obliczeń dostępny jest także przycisk zatrzymania obliczeń, a aktualny stan symulacji jest podmieniany dopiero po poprawnym zakończeniu liczenia nowych klatek.

8 Porównanie symulacji z rozwiązaniem analitycznym

W celu weryfikacji poprawności działania zaimplementowanych algorytmów całkowania zależnego od czasu równania Schrödingera przeprowadzono testy porównawcze wyników symulacji numerycznych ze ścisłymi rozwiązaniami analitycznymi. Proces ten został w pełni zautomatyzowany w ramach potoku ciągłej integracji.

8.1 Rozwiązania analityczne

W przypadku nieskończonej studni potencjału istnieją rozwiązania analityczne w zamkniętej formie, których można użyć do sprawdzenia wyników algorytmów i ich zbieżności przy zmniejszaniu kroku czasowego. Rozwiązania te dane są przez kombinacje liniowe stanów własnych hamiltonianu:

$$|n_x, n_y\rangle = \sqrt{\frac{2}{L_x}} \sin\left(\frac{\pi n_x}{L_x} x\right) \sqrt{\frac{2}{L_y}} \sin\left(\frac{\pi n_y}{L_y} y\right).$$

Dla skończonego $A \subset \mathbb{N}^2$, funkcja falowa dana początkowo przez

$$\Psi_A(t=0) = \sum_{(n_x, n_y) \in A} C_{n_x, n_y} |n_x, n_y\rangle$$

ewoluuje zgodnie z

$$\Psi_A(t) = \sum_{(n_x, n_y) \in A} C_{n_x, n_y} e^{-iE_{n_x, n_y}t/\hbar} |n_x, n_y\rangle,$$

gdzie $E_{n_x, n_y} = \frac{\hbar^2}{2m} \left(\frac{n_x^2}{L_x^2} + \frac{n_y^2}{L_y^2} \right)$.

8.2 Metodologia walidacji i metryki błędów

Analizę zbieżności oparto na wyznaczeniu błędu $\Delta(\mathbf{r}, \delta t)$, będącego różnicą pomiędzy numeryczną funkcją falową Ψ_{num} a rozwiązaniem wzorcowym Ψ_{analit} dla zadanego kroku czasowego δt :

$$\Delta(\mathbf{r}, \delta t) = \Psi_{\text{num}}(\mathbf{r}, \delta t) - \Psi_{\text{analit}}(\mathbf{r}, \delta t) \quad (27)$$

Do ilościowej oceny dokładności solverów wprowadzono dwie metryki błędów:

- **maksymalny błąd lokalny** (E_{sup}) w normie supremum:

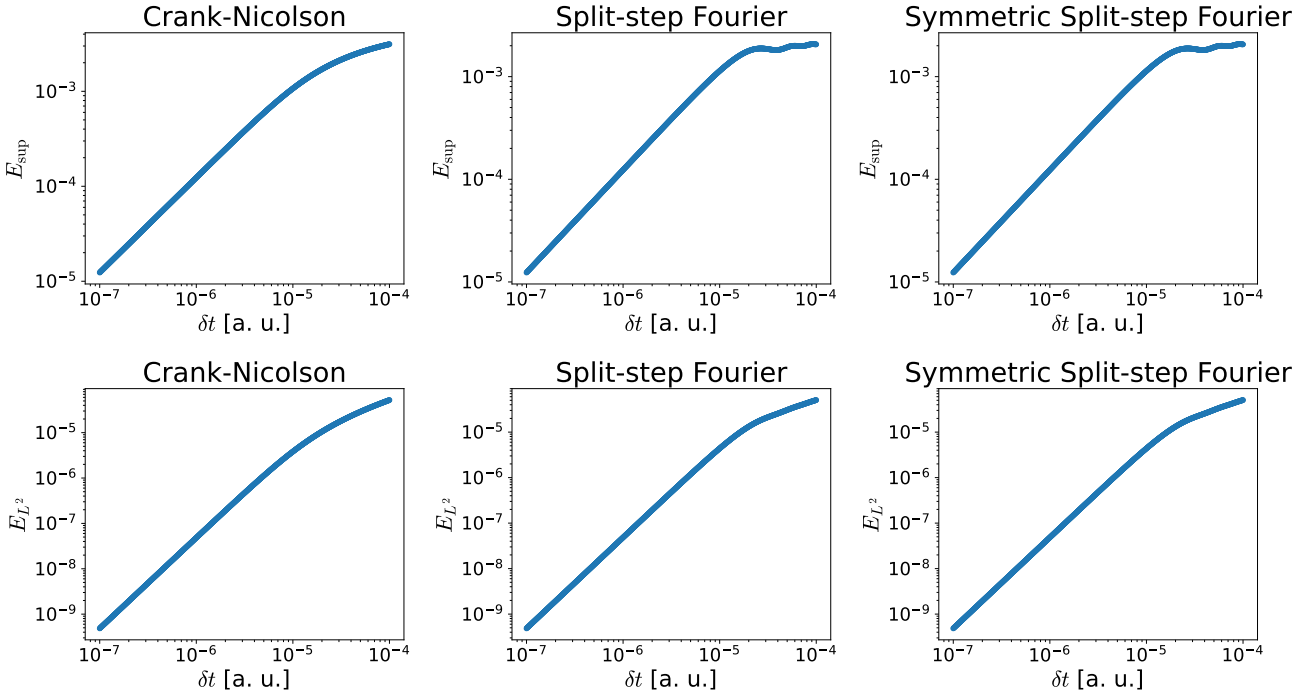
$$E_{\text{sup}} = \sup_{\mathbf{r} \in \Omega} |\Delta(\mathbf{r}, \delta t)| \quad (28)$$

- **globalny błąd średniokwadratowy** (E_{L^2}) w normie L^2 :

$$E_{L^2} = \iint_{\Omega} |\Delta(\mathbf{r}, \delta t)|^2 d^2r \quad (29)$$

8.3 Analiza zbieżności schematów numerycznych

Badanie zależności wskaźników E_{sup} oraz E_{L^2} od kroku czasowego δt przeprowadza skrypt `step_dependence.py`. Testom poddano trzy metody propagacji: schemat Cranka-Nicolson, standardową metodę Split-Step Fourier oraz jej wariant symetryczny.



Rysunek 2: Zależność błędów E_{sup} i E_{L^2} od kroku czasowego δt dla analizowanych metod całkowania.

Wyniki potwierdzają stabilność oraz teoretyczne rzędy zbieżności algorytmów:

$$\lim_{\delta t \rightarrow 0} E_{\text{sup}} = 0, \quad \lim_{\delta t \rightarrow 0} E_{L^2} = 0 \quad (30)$$

Dla schematu Cranka-Nicolson oraz symetrycznej metody Split-Step Fourier nachylenie krzywych błędów w skali podwójnie logarytmicznej wskazuje na drugą klasę dokładności względem czasu.

8.4 Automatyzacja procesów testowych

W celu zapewnienia stabilności kodu, proces weryfikacji numerycznej i testów jednostkowych został w pełni zautomatyzowany przy użyciu platformy GitHub Actions. Zgodnie z konfiguracją zawartą w pliku `tests.yml`, przy każdym zdarzeniu wypchnięcia zmian do repozytorium, automatycznie uruchamiane jest odizolowane środowisko systemowe.

Infrastruktura testowa realizuje sekwencyjnie następujące kroki:

1. Pobranie aktualnej wersji kodu źródłowego.
2. Instalacja menedżera pakietów `uv` wraz z buforowaniem zależności.
3. Konfiguracja środowiska Python oraz instalacja wymaganych pakietów matematycznych.
4. Wykonanie bazy testów z wykorzystaniem frameworka `pytest`.

9 Repozytorium kodu oraz dalszy rozwój

Repozytorium kodu znajduje się pod adresem github.com/Hansus00/Quaint. Pomysły na przyszły rozwój oraz uwagi i zgłoszenia błędów znajdują się pod adresem [Quaint/issues](https://github.com/Hansus00/Quaint/issues).

Bibliografia

- [1] Charles R. Harris i in. „Array programming with NumPy”. W: *Nature* 585.7825 (2020), s. 357–362. DOI: [10.1038/s41586-020-2649-2](https://doi.org/10.1038/s41586-020-2649-2).
- [2] Pauli Virtanen i in. „SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. W: *Nature Methods* 17 (2020), s. 261–272. DOI: [10.1038/s41592-019-0686-2](https://doi.org/10.1038/s41592-019-0686-2).
- [3] J. D. Hunter. „Matplotlib: A 2D graphics environment”. W: *Computing in Science & Engineering* 9.3 (2007), s. 90–95. DOI: [10.1109/MCSE.2007.55](https://doi.org/10.1109/MCSE.2007.55).
- [4] Z. Sun i W. Yang. „Communication: An exact short-time solver for the time-dependent Schrödinger equation”. W: *The Journal of Chemical Physics* 134.4 (2011), s. 041101. DOI: [10.1063/1.3549570](https://doi.org/10.1063/1.3549570).
- [5] Wai Kui Wong. *Solving 2D Time Independent Schrodinger Equation Using Numerical Method*. Grud. 2021. DOI: [10.13140/RG.2.2.12835.99360](https://doi.org/10.13140/RG.2.2.12835.99360).
- [6] J. Crank i P. Nicolson. „A practical method for numerical evaluation of solutions of partial differential equations of the heat-conduction type”. W: *Mathematical Proceedings of the Cambridge Philosophical Society* 43.1 (1947), s. 50–67. DOI: [10.1017/S0305004100023197](https://doi.org/10.1017/S0305004100023197).
- [7] Thiab R Taha i Mark I Ablowitz. „Analytical and numerical aspects of certain nonlinear evolution equations. II. Numerical, nonlinear Schrödinger equation”. W: *Journal of Computational Physics* 55.2 (1984), s. 203–230. DOI: [10.1016/0021-9991\(84\)90003-2](https://doi.org/10.1016/0021-9991(84)90003-2).
- [8] Masuo Suzuki. „General theory of higher-order decomposition of exponential operators and symplectic integrators”. W: *Physics Letters A* 165.5 (1992), s. 387–395. DOI: [https://doi.org/10.1016/0375-9601\(92\)90335-J](https://doi.org/10.1016/0375-9601(92)90335-J).
- [9] Elliott S. Wise i in. „Pseudospectral Time-Domain (PSTD) Methods for the Wave Equation: Realizing Boundary Conditions with Discrete Sine and Cosine Transforms”. W: *Journal of Theoretical and Computational Acoustics* 29.04 (2021), s. 2050021. DOI: [10.1142/S2591728520500218](https://doi.org/10.1142/S2591728520500218).
- [10] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations*. 2 wyd. Society for Industrial i Applied Mathematics, 2007. Rozd. 13.2. DOI: [10.1137/1.9780898717839](https://doi.org/10.1137/1.9780898717839).

A Podręcznik użytkownika w języku angielskim

Quaint

Interactive Quantum Wavepacket Simulator

User Manual

Jacek Winiarczyk Szymon Borowski Franciszek Hansdorfer
Bartosz Łakomy Adam Podkowiński

Faculty of Physics, University of Warsaw

This project was developed under the mentorship of mgr Bartosz Kasza

June 30, 2026

Contents

1	Installation	2
1.1	Basic Installation	2
1.2	Linux Troubleshooting (Qt & xcb errors)	2
2	Getting Started	3
3	Animation Window	5
3.1	Camera Controls	5
3.2	Playback Controls	5
4	Simulation Setup Window	7
4.1	Painting the Potential	7
4.2	Preset Potentials	8
4.3	Setup Options	8
4.3.1	Simulation Settings	9
4.3.2	Grid Domain	9
4.3.3	Wavepacket Parameters	10
4.4	Saving and Loading Configurations	10
5	Visual Settings	10

1 Installation

Quaint requires Python 3.10 or newer. To ensure a smooth installation of all dependencies, we strongly recommend using `uv`, a lightning-fast Python package installer and environment manager.

1.1 Basic Installation

Open your terminal and execute the following commands step by step to clone the repository and launch the application:

- Clone the repository:

```
git clone https://github.com/your-username/
quaint.git
```

- Navigate to the project directory:

```
cd quaint
```

- Install dependencies and create a virtual environment:

```
uv sync
```

- Activate the virtual environment:

```
source .venv/bin/activate
```

- Run the application:

```
python main.py
```

1.2 Linux Troubleshooting (Qt & xcb errors)

Quaint utilizes PyQt6 for its graphical user interface. On Linux distributions (such as Ubuntu, Mint, or Debian), standard installations often lack the low-level windowing libraries required by Qt.

If the application crashes on startup with an error mentioning `qt.qpa.plugin: Could not load the Qt platform plugin "xcb"`, you need to install the missing system libraries. Run the following command to install the most common PyQt6 dependencies:

```
sudo apt update
```



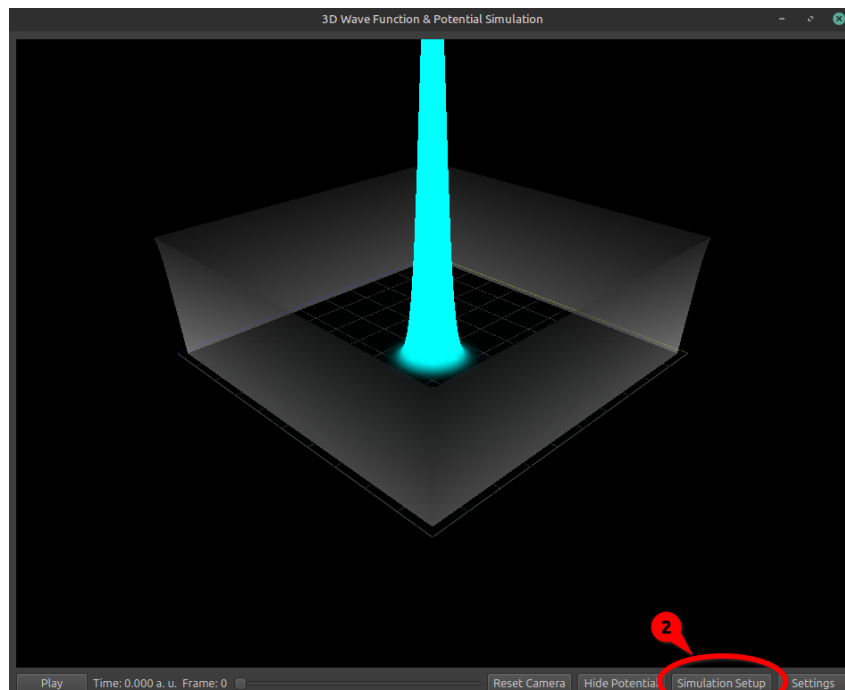
```
sudo apt install libxcb-cursor0 libxkbcommon-x11-0 \
    textbackslash
libxcb-xinerama0 libxcb-icccm4 \textbackslash
libxcb-image0 libxcb-keysyms1 \textbackslash
libxcb-randr0 libxcb-render-util0 \textbackslash
libxcb-shape0
```

After installation, try running `python main.py` again.

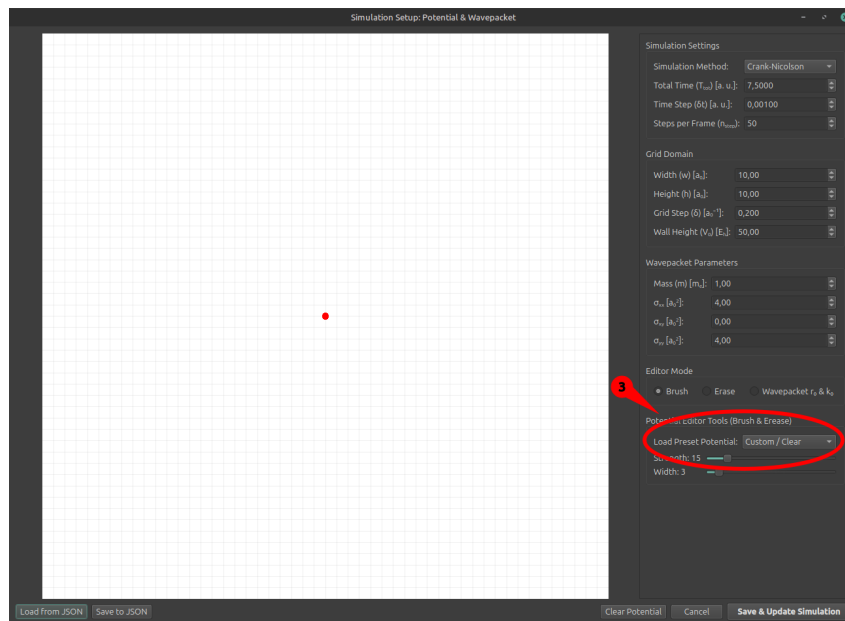
2 Getting Started

Running your first quantum simulation is straightforward:

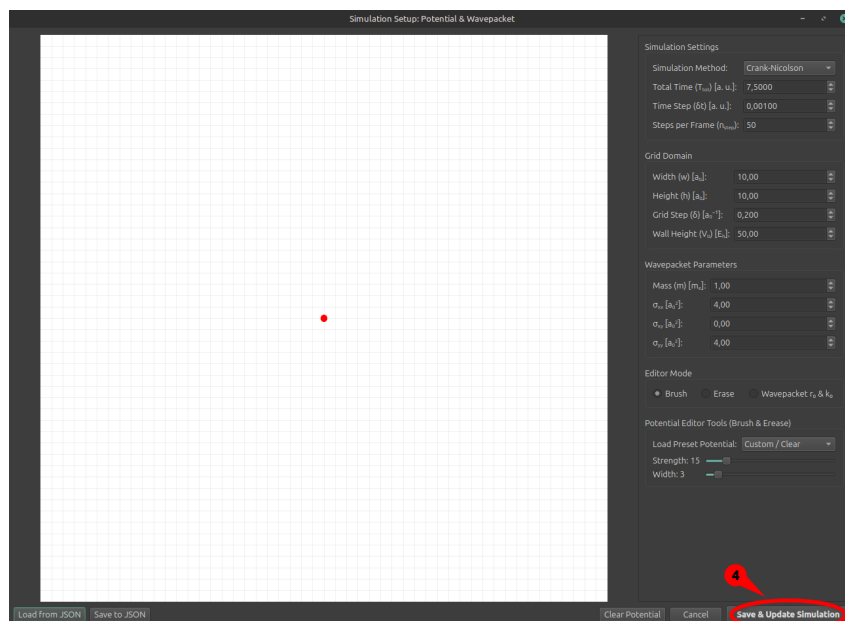
1. Launch Quaint. The main 3D visualization window will appear.
2. Click the **Simulation Setup** button located at the bottom of the screen.



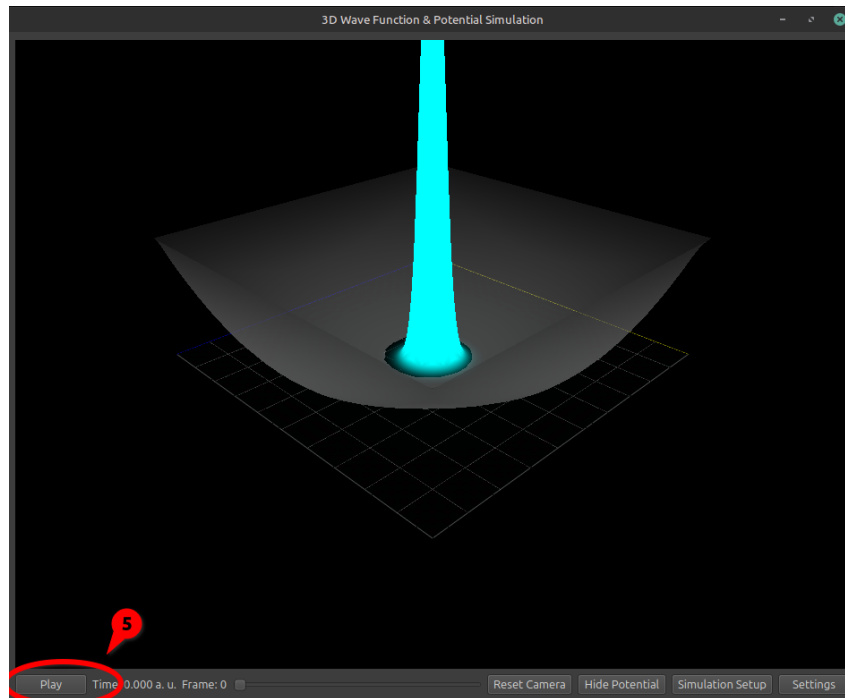
3. In the right-hand panel of the setup window, locate the **Load Preset Potential** dropdown. Select *Harmonic Oscillator*.



4. Click **Save & Update Simulation** at the bottom right.



5. Once the calculation is complete, press the **Play** button (or the Spacebar) to watch the wavepacket evolve in the potential well.



3 Animation Window

The main window is dedicated to the real-time, 3D OpenGL visualization of the wavepacket and the underlying physical potential landscape.

3.1 Camera Controls

The simulation is rendered in full 3D space. You can navigate the view freely using your mouse:

- **Rotate:** Left-click and drag the mouse across the canvas.
- **Zoom:** Right-click and drag (up/down) or use your mouse scroll wheel.
- **Pan:** Middle-click (scroll wheel click) and drag to move the camera focus point.

3.2 Playback Controls

Located at the bottom of the screen, the playback bar allows you to traverse the calculated simulation frames.

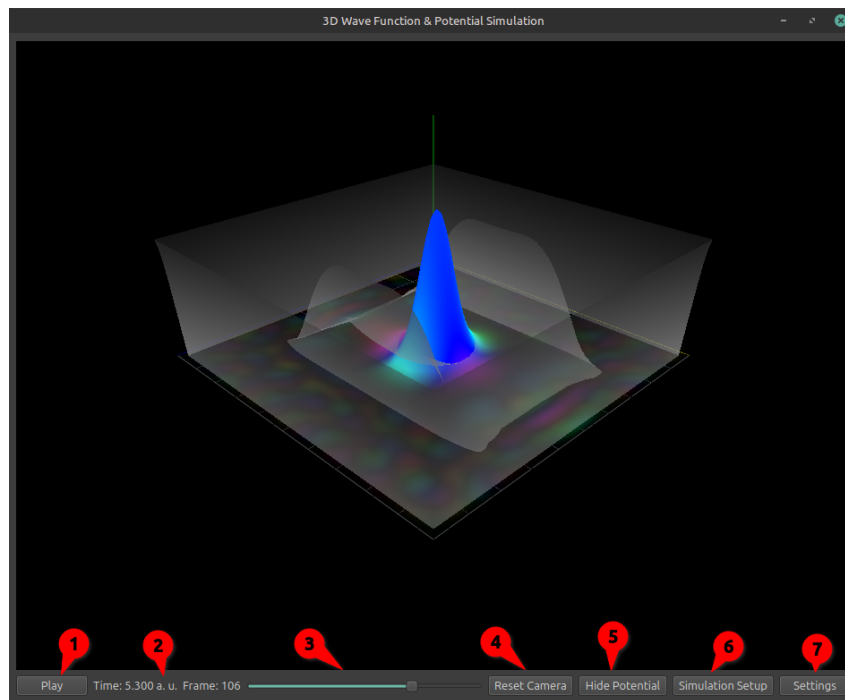


Figure 1: Main 3D animation window.

- **Play / Pause (1):** Toggles the animation playback. Alternatively, you can use the **Space-bar** as a global shortcut.
- **Time Label (3):** Displays the current physical time elapsed in atomic units (a.u.) and the current frame index.
- **Timeline Slider (2):** Click and drag the slider to scrub through the animation manually.
- **Reset Camera (4):** Instantly snaps the 3D camera back to its default viewing angle and distance.
- **Hide / Show Potential (5):** Toggles the visibility of the semi-transparent 3D potential mesh. Hiding it can give you a clearer, unobstructed view of the complex wave function.
- **Simulation Setup (6):** Opens the setup drawer, pausing the simulation and allowing you to redefine physical parameters and the environment.
- **Settings (7):** Opens a dedicated window for visual configuration (e.g., rendering scales, brightness).
- **Stop Calculation:** This button only appears while the backend is computing a new simulation. Clicking it safely aborts the heavy calculation without crashing the program, allowing you to return to the setup window.

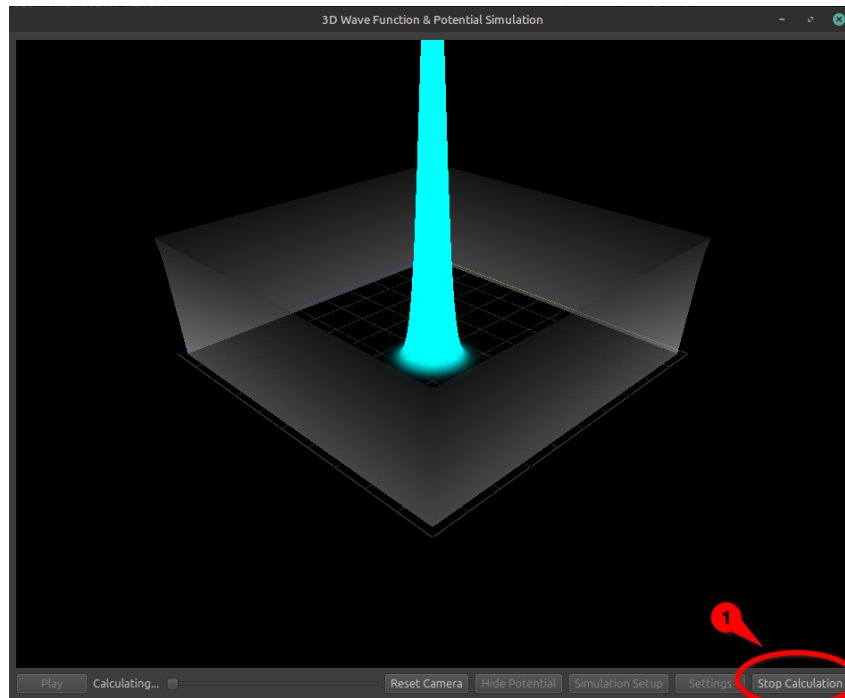


Figure 2: Main 3D animation window in calculation mode.

4 Simulation Setup Window

The Setup Drawer is the core of Quaint. It is a comprehensive physics configuration interface linking your manual drawings to the mathematical simulation parameters.

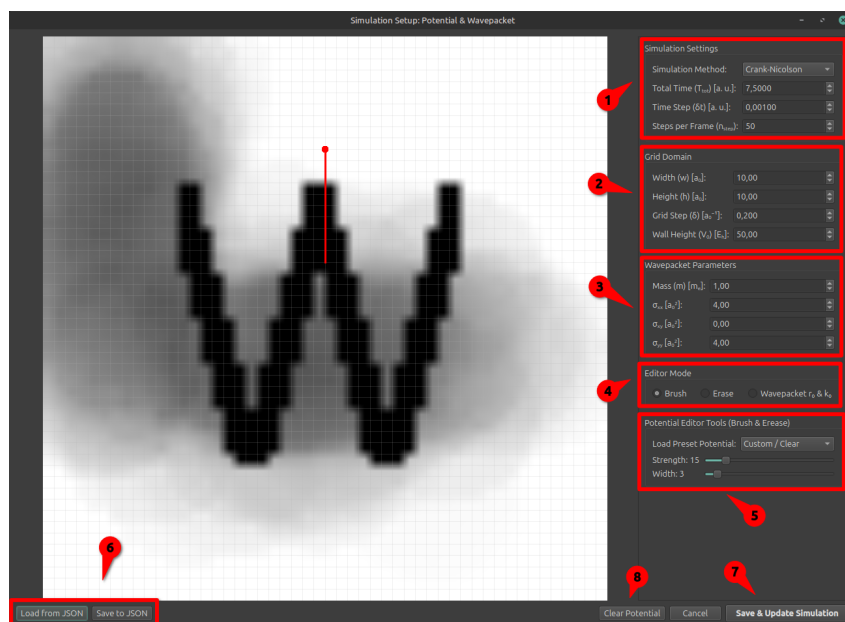


Figure 3: The Simulation Setup interface.

4.1 Painting the Potential

The left side of the window is an interactive canvas representing your 2D spatial grid.

Note

The simulation domain is strictly bounded by infinite potential walls. Any wavepacket reaching the edge of the canvas will reflect back. If you wish to simulate free space for a longer duration, you must increase the physical dimensions of the grid.

You can modify the potential landscape using the **Editor Mode (4)** tools:

- **Brush:** Click and drag to paint potential barriers.
- **Erase:** Click and drag to remove previously drawn potential barriers, resetting those specific grid cells to zero potential.
- **Wavepacket r_0 & k_0 :** Sets the initial state of the quantum particle. Click once on the canvas to anchor the initial position (r_0), then drag outward and release to define the initial momentum vector (k_0). The length and direction of the drawn arrow dictate the speed and trajectory of the packet.

To gain finer control over the drawing tools, adjust the sliders in the **Potential Editor Tools (5)** section:

- **Strength Slider:** Adjusts the height of the drawn potential. A value of 100 corresponds to the maximum wall height defined in your grid settings, while lower values create softer barriers.
- **Width Slider:** Changes the diameter of the brush (and eraser) in grid cells.

4.2 Preset Potentials

Instead of drawing by hand, you can automatically generate precise, mathematical potential landscapes using the **Load Preset Potential** dropdown from the **Potential Editor Tools (5)** section. Selecting any of the presets will immediately overwrite the canvas with the corresponding potential shape and reset the wavepacket to its default position for that scenario:

- **Custom / Clear:** Wipes the canvas clean to a flat, zero-potential state.
- **Gaussian Bump:** Generates a smooth, central potential hill.
- **Harmonic Oscillator:** Creates a classic parabolic well originating from the center of the grid, with the wavepacket in the center by default.
- **W-shape:** Forms an intricate double-well potential structure. The preset automatically drops the wavepacket near the top center with a downward momentum.
- **Matryoshka:** Creates a scaled down version of the current canvas state in the center, surrounded by potential walls.
- **Slab:** A simple vertical barrier dividing the grid into two halves.

4.3 Setup Options

The right panel contains precise numerical parameters defining the physics engine behavior.

4.3.1 Simulation Settings

The **Simulation Settings (1)** section allows you to configure the core numerical parameters of the time evolution algorithm and the simulation method.

- **Simulation Method:** Choose the numerical algorithm for time evolution. *Crank-Nicolson* is unconditionally stable and highly accurate, but can be slower on very large grids. *SSFM* (Split-Step Fourier Method) and *Symmetric SSFM* are typically faster for large domains but require periodic boundaries.
- **Total Time (T_{tot}):** The total physical duration of the simulation in atomic units.
- **Time Step (δt):** The size of each mathematical integration step. Smaller values increase accuracy but require more calculations.
- **Steps per Frame (n_{step}):** The number of background simulation steps calculated before capturing a single frame for the animation. Increasing this speeds up the visual playback of long simulations without losing mathematical accuracy.

Warning

Numerical Stability: Quantum simulations are highly sensitive to initial conditions. If your Time Step (δt) is too large relative to the Grid Step (δ), the numerical solver may diverge.

Quaint features an automatic stability checker. Instead of crashing or producing broken "exploding" artifacts, Quaint will intercept unstable parameters, display a detailed warning, and safely return you to the Setup window so you can lower the time step or adjust the grid.

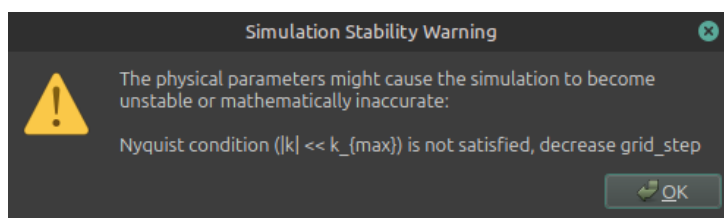


Figure 4: A stability warning pop-up.

4.3.2 Grid Domain

The **Grid Domain (2)** parameters define the physical dimensions and resolution of your simulation space.

- **Width (w) & Height (h):** The physical dimensions of the simulation space, measured in Bohr radii (a_0).
- **Grid Step (δ):** The distance between individual calculation nodes.

Note

Smaller steps provide significantly higher spatial resolution, but exponentially increase memory usage and computation time. If your simulation takes too long to generate or the interface lags, increasing the Grid Step is the most effective way to speed it up.

- **Wall Height (V_0):** The maximum potential energy value in the simulation, measured in Hartrees (E_h). This value acts as the ceiling for your Brush tool.

4.3.3 Wavepacket Parameters

The **Wavepacket Parameters (3)** allow you to define the initial quantum state of the particle before time evolution begins. The wavepacket is mathematically represented as a 2D Gaussian function, and these parameters control its shape and spread.

- **Mass (m):** The mass of the simulated particle, expressed in units of electron mass (m_e).
- σ_{xx} & σ_{yy} : The spatial uncertainty (spread) of the initial wavepacket along the X and Y axes, measured in squared Bohr radii (a_0^2). Larger values create a wider, more diffuse initial packet.
- σ_{xy} : The correlation between the X and Y uncertainties, allowing you to create diagonally skewed wavepackets.

4.4 Saving and Loading Configurations

Achieved a fascinating quantum simulation? You can preserve your setup using the buttons in the bottom left corner (6).

- **Save to JSON:** Exports your entire canvas drawing, potential height, and all physical parameters to a lightweight JSON file.
- **Load from JSON:** Imports a previously saved configuration, immediately overwriting the current canvas and settings.

Once you are satisfied with all parameters, click **Save & Update Simulation (7)** to build the matrices and begin the heavy calculation process.

5 Visual Settings

Clicking the **Settings** button in the main window opens a non-modal dialog dedicated strictly to rendering aesthetics. Modifying these values updates the 3D view instantly *without* forcing a simulation recalculation.

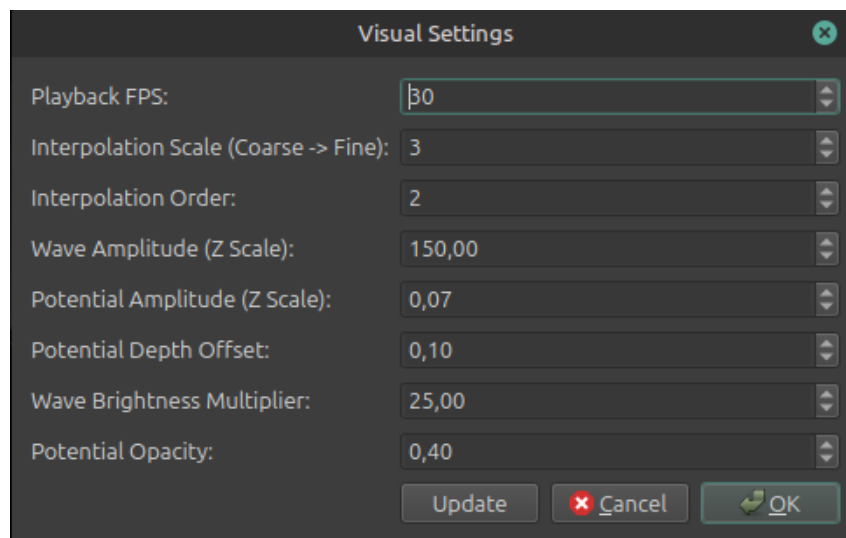


Figure 5: Visual settings dialog.

- **Playback FPS:** Adjusts the target frames-per-second rate of the animation player.
- **Interpolation Scale:** The application renders the wave function on a visually finer grid than the one used for actual physics calculations. This value dictates how many visual cells correspond to one physical grid cell.
- **Interpolation Order:** Controls the B-spline math used during upscaling. Order 1 is fast but faceted; Order 2 is the recommended sweet spot; Order 3+ is very smooth but computationally heavier and might artificially overshoot near sharp peaks.
- **Wave Amplitude (Z Scale):** A purely visual multiplier that stretches the probability peaks upwards, making them easier to distinguish from the floor.
- **Potential Amplitude (Z Scale):** Vertically stretches the 3D potential mesh.
- **Potential Depth Offset:** Physically shifts the entire potential mesh up or down along the Z-axis to prevent it from clipping through the wave function.
- **Wave Brightness Multiplier:** Increases the color exposure of the wave graph. This is useful for visualizing extremely faint probability tails that have tunneled through barriers.
- **Potential Opacity:** Controls the transparency level (Alpha channel) of the potential landscape.

B Podział prac w zespole

Szymon Borowski – frontend, m.in.:

- implementacja odczytu i zapisu parametrów symulacji,
- wielowątkowość: wydzielenie asynchronicznego wątku do obliczania ramek symulacji,
- integracja z backendem (pobieranie zdefiniowanych kształtów potencjałów oraz stanów symulacji).

Adam Podkowiński – frontend, m.in.:

- rendering 3D funkcji falowej oraz profilu potencjału,
- implementacja interaktywnego obszaru roboczego (canvas) do manualnego generowania potencjałów,
- projekt interfejsu (panel ustawień, kontrolki sterujące symulacją).

Jacek Winiarczyk – backend, m.in.:

- implementacja schematu Cranka-Nicolson oraz metody Cayleya,
- opracowanie uniwersalnego modułu testującego oraz mechanizmu zapisu wyników do zoptymalizowanego formatu JSON,
- przegląd kodu (code review) i audyt użyteczności UI/UX,
- narzędzia do weryfikacji rozwiązań numerycznych z modelami analitycznymi.

Bartosz Łakomy – backend, m.in.:

- współimplementacja schematu Cranka-Nicolson i metody Cayleya,
- projektowanie fizycznych profili potencjałów,
- implementacja analitycznych rozwiązań wzorcowych,
- moduły wyznaczające parametry fizyczne, m.in. oczekiwaną wartość energii $\langle E \rangle$ oraz całkowite prawdopodobieństwo $P(t)$.

Franciszek Hansdorfer – backend, m.in.:

- opracowanie i wdrożenie metod podziału kroku (SSFM oraz wariantu symetrycznego),
- administracja repozytorium oraz systemem kontroli wersji,
- utrzymanie wysokiej jakości kodu, egzekwowanie dobrych praktyk programistycznych oraz weryfikacja anglojęzycznej nomenklatury,
- przegląd kodu (code review).